

CameraReady: A Gated, Verification-First Workflow for Agentic Academic Paper Production

Yanwei Liu
Independent Researcher

Abstract—While large language model (LLM) agents can automate end-to-end scientific writing, they frequently fabricate citations and assert unsubstantiated empirical claims. Existing defenses operate predominantly post-hoc, scanning finished drafts to catch spurious references after text generation. In this technical report, we argue that mechanically checkable manuscript-integrity properties are better enforced by the authoring workflow than delegated to model self-assessment. We present CameraReady, an open-source, verification-first authoring framework that guides scientific papers from initial ideation to an arXiv-oriented pre-submission package across ten structured phases. CameraReady structures verification into a three-tier taxonomy: enforced filesystem invariants that halt progression upon failure, advisory diagnostics that flag potential semantic or stylistic issues for author review, and explicit human governance checkpoints. The citation pipeline decouples parametric memory from the bibliography: references enter exclusively via external API resolution against canonical registries (DBLP, CrossRef, and arXiv), ensuring that unresolved queries produce explicit verification markers rather than ungrounded BibTeX records. Discipline conventions, citation styles, and venue formatting constraints are formalized as declarative data schemas rather than prompt instructions. Under the stated registry-resolution criterion, all 45 references ingested for this case study originated from canonical registry queries, with no identity-level fabricated bibliographic records observed in the resulting bibliography. Repeated downstream re-resolution under live network conditions exhibited transient remote-service throttling (yielding 38–45 verified entries across runs), illustrating an operational advantage of persisting bibliographic provenance at ingestion time rather than relying solely on retrospective remote lookup. Testing 545 unit tests in 34 modules, together with negative-control fault injections and an end-to-end dogfooding case study, showed that all blocking gates rejected the tested non-compliant inputs with non-zero exit codes; controlled benchmark comparison against unconstrained authoring remains future work. CameraReady is open source at <https://github.com/e96031413/camera-ready>.

Index Terms—LLM agents, scientific writing, citation hallucination, workflow verification, research software

I. INTRODUCTION

Citations form the foundation of scholarly literature: cited prior work must exist and directly support the citing text. Generative language models routinely violate this invariant. In an audit of 111 million citations across arXiv, bioRxiv, SSRN, and PubMed Central, non-existent references rose sharply alongside LLM adoption [1]. Fabricated citations have begun slipping into published conference proceedings [2], and

systematic evaluations confirm that citation fabrication persists across all major commercial and open-weight models [3].

Concurrently, autonomous research agents have expanded from simple code execution to open-ended discovery. Frameworks such as The AI Scientist [4] and Agent Laboratory [5] automate ideation, experimentation, and paper compilation. Subsequent systems scale agentic research to multi-day lifecycles [6], [7] and distributed teams [8], [9]. Structured generation pipelines enforce argument outlines prior to drafting [10], while broader surveys document rapid adoption across scientific domains [11]–[13]. However, manuscript integrity in these frameworks is predominantly relegated to downstream evaluation benchmarks [14] or retrospective post-hoc screening [15], [16].

Post-hoc filtering exhibits fundamental architectural weaknesses. Retrospective checkers inspect drafts only after spurious text has been generated, relying on empirical similarity thresholds and third-party web APIs that remain vulnerable to rate limits and network outages. As Cornelio et al. [17] argue, verification must be embedded directly into the research and authoring loop. Parallel initiatives have introduced auditable protocols for scientific hypothesis formation [18], [19]. However, existing agent frameworks lack fail-stop verification gates, deterministic citation grounding, and auditable repository predicates.

This report presents CameraReady, an open-source workflow comprising 68 modular Python scripts (≈ 21.2 k lines of code) guiding manuscripts from ideation to a pre-submission package passing local format and integrity gates. CameraReady is grounded on a clear principle: model-generated completion claims are insufficient; phase transitions must be determined from repository-level predicates. Operating under a cooperative agent model where toolchain execution governs repository state, the framework partitions verification into three explicit operational tiers: 1) *Enforced invariants*: mechanically decidable checks (canonical bibliographic existence, numerical consistency against registered data artifacts, schema conformance, and prerequisite artifact existence) halting workflow progression via fail-stop exit codes; 2) *Advisory diagnostics*: semantic and stylistic heuristics (citation lexical overlap, prose flow, and pattern density) assisting author inspection without blocking compilation; and 3) *Human governance*: scholarly judgment, scientific novelty, claim adjudication, and submission authorization. References enter exclusively through API-mediated resolution against canonical registries, and venue formatting constraints reside in structured configuration schemas.

This document is an engineering technical report accompanying CameraReady (release v0.2.0); it has not undergone formal peer review. Source code, evaluation records, citation provenance manifest, and reproduction scripts: <https://github.com/e96031413/camera-ready>.

Our primary contributions are fourfold: 1) *Three-tier gated workflow* (Section VI): A state machine governing ten phases through fail-stop invariant gates, advisory diagnostics, and append-only override journaling; 2) *Citation provenance by construction* (Section VII): A bibliographic pipeline that decouples parametric memory from `ref.bib`, admitting entries only via direct API resolution against DBLP, CrossRef, or arXiv; 3) *Provided declarative schemas* (Section V): Formalized configuration profiles for 8 disciplines, 6 citation styles, and 6 venues, enabling uniform verification across distinct publishing standards; and 4) *Artifact-level verification and provenance analysis* (Section VIII): Software verification across 545 unit tests (across 34 modules), 12 negative control fault injections, and an operational comparison demonstrating the stability of fetch-time provenance persistence over downstream remote API re-resolution under rate limits; controlled empirical comparisons against unconstrained agent baselines remain future work.

II. BACKGROUND AND RELATED WORK

A. Autonomous research and writing agents

LLM-based scientific agents have advanced from isolated coding assistants to autonomous discovery pipelines. Lu et al. [4] demonstrated fully automated research loops closed by automated peer review. Agent Laboratory [5] partitions tasks among specialized agents for literature review, experimentation, and report compilation. Long-horizon discovery systems such as ScienceFlow [6], Kosmos [7], and FARS [8] support distributed multi-agent collaboration, while goal-evolving architectures [9] dynamically adjust hypotheses. In manuscript preparation, Story2Proposal [10] emphasizes structural outlines, while broader surveys document rapid adoption across scientific domains [11]–[13]. Wang et al. [20] and Takahara et al. [18] advocate externalizing research validation into auditable execution environments. CameraReady applies this externalization directly to the authoring pipeline, substituting model self-evaluation with deterministic filesystem inspection.

B. Citation hallucination: measurement and detection

The prevalence of citation hallucination is well documented. Zhao et al. [1] estimate that tens of thousands of fabricated citations have entered scholarly preprints, while Naser [3] demonstrates that hallucination is widespread across both commercial and open-weight LLMs. Mechanistic studies by Chen et al. [21] trace citation fabrication to internal parametric representations, confirming that hallucination is an architectural consequence of autoregressive modeling.

In response, post-hoc detectors verify citations via retrieval-augmented search (CiteCheck [16]), internal attribution tracing (FACTUM [22]), existential queries (CheckIfExist [23]), and multi-agent cross-examination [24]. Rao et al. benchmark citation errors in publishing agents [25] and evaluate automated correction [15], while MCP servers provide interactive bibliographic repair [26]. However, grounding alone cannot eliminate invented citations [27]. Unlike retrospective detectors

that inspect drafts after errors occur, CameraReady enforces bibliographic provenance prior to drafting.

C. Verification and provenance in agent architectures

Agent systems increasingly incorporate external verification. CARE [28] statically checks shell commands before execution; TRACER [29] provides cryptographic provenance for multimodal outputs; and She et al. [30] track provenance to detect policy drift. Souza et al. [31] formalize reference architectures for workflow provenance. For iterative loops, Wu et al. [32] formulate bounded stopping rules, and Sah et al. [33] model the “verifier tax” governing safety-throughput trade-offs. Tool evolution studies [34] and programmatic skill workflows [35] similarly replace unconstrained model autonomy with structured execution.

D. Review, detection, and reporting norms

Automating scientific review introduces distinct risks [36], [37], as machine-generated text increasingly permeates both submissions [38] and reviewer comments [39]. While span classifiers attempt to detect synthetic prose [40], formal reproducibility checklists [41] and automated code audits [42] provide objective reporting criteria. Systematic review pipelines [43]–[45] also require strict reporting compliance. CameraReady formalizes these standards as machine-readable schemas, focusing strictly on verifiable structural, bibliographic, and procedural properties.

III. THREAT MODEL

We consider an LLM agent that operates through standard shell and filesystem interfaces and is tasked with authoring an academic paper targeting a peer-reviewed venue. Assuming the agent operates cooperatively, failure stems not from malice, but from generative unreliability. We identify four primary failure modes:

F1: Fabricated or corrupted citations. The agent emits syntactically valid BibTeX referencing nonexistent papers or attributing real authors to invented titles [25]. Autoregressive sampling cannot reliably separate recall from fabrication.

F2: Claims unsupported by evidence. The manuscript asserts findings or performance metrics unsupported by underlying experimental artifacts, or cites literature for techniques they do not contain.

F3: Formatting and venue policy violations. The generated document violates page limits, required sections, style guidelines, or double-blind anonymization rules.

F4: Premature workflow progression. The agent declares a phase complete based on conversational cues rather than verified artifact readiness, leading to ungrounded drafting.

CameraReady deterministically enforces the **mechanically checkable subset** of F1–F4: canonical bibliographic existence and registry resolution (F1 subset), numerical consistency between reported tables and registered script outputs (F2 subset), adherence to declarative venue formatting schemas (F3), and physical existence of phase prerequisite artifacts (F4). In contrast, higher-order semantic validity—such as

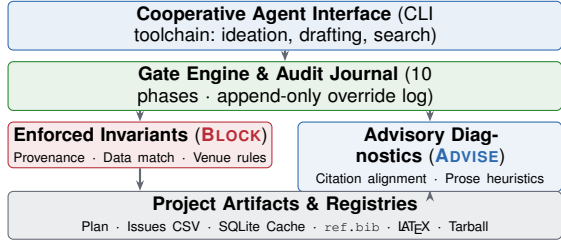


Fig. 1. System architecture and verification tiers: The gate engine enforces mechanically checkable invariants (BLOCK), presents diagnostic heuristics (ADVISE), and preserves audit trails across physical project artifacts.

whether a cited paper genuinely substantiates a nuanced conceptual assertion, or whether an experimental methodology is sound—cannot be decided mechanically; such properties remain advisory diagnostics or explicit human responsibilities. **Operating model and trust boundary.** We assume an agent executing commands cooperatively within the CameraReady CLI and toolchain workflow. CameraReady establishes toolchain invariants ensuring that compliant execution cannot silently inject ungrounded citations or bypass unfulfilled phase prerequisites. We do not model an adversarial agent actively attempting to tamper with repository state via out-of-band shell commands; adversarial isolation requires OS-level sandboxing (discussed in Section IX).

IV. DESIGN PRINCIPLES

CameraReady addresses failures F1–F4 through five design principles:

P1: Externalized, file-backed state. Execution state resides in structured filesystem artifacts (JSON, Markdown) rather than ephemeral conversational memory, allowing workflows to pause and resume across sessions without state loss.

P2: Fail-stop gating by default. Verification gates exit with non-zero status codes upon invariant failure; 62 of 68 scripts implement this fail-stop behavior. Advancing requires satisfying all checks or explicitly documenting an override. Advisory diagnostics exit cleanly with warnings.

P3: Strict provenance within the toolchain. Within the supported CLI, the pipeline provides no code path for models to inject raw BibTeX into `ref.bib`. Entries are admitted exclusively via automated API resolution against canonical registries; unresolved queries emit explicit markers.

P4: Declarative discipline and venue schemas. Field guidelines, citation conventions, and venue constraints are formalized as declarative data schemas rather than natural-language prompt instructions.

P5: Explicit human authority checkpoints. Critical governance boundaries—specifically approving the initial research plan, adjudicating extracted claims, and authorizing final submission—are enforced as explicit, file-backed sign-off checkpoints within the operating model.

V. SYSTEM ARCHITECTURE

CameraReady is organized into four architectural layers (Fig. 1): declarative rules, the gate engine, modular verification checkers, and physical project artifacts.

TABLE I
PROVIDED CONFIGURATION PROFILES AND SPECIFICATION SCHEMAS

Dimension	Count	Configured Profiles and Validation Status
Disciplines	8	CS (exercised); Medicine, Psych., Educ., Bus., Soc. Sci., Hum., Law (schema)
Styles	6	IEEE (exercised); APA 7, MLA 9, Vancouver, Chicago (schema)
Venues	6	NeurIPS, ICML, ICLR, ACL, AAAI, CVPR (CFP schema)
Formats	4	LATEX (native); DOCX, Markdown, Typst (via Pandoc)

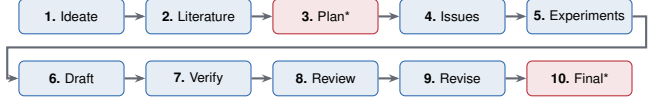


Fig. 2. The ten workflow phases (* indicates mandatory human-in-the-loop sign-off; red phases cannot close autonomously; non-linear re-entry is supported).

A. Rules as declarative data

Discipline profiles are formalized as YAML documents defining mandatory sections, citation conventions, reporting standards, and common domain rejection pitfalls. For example, the computer science profile mandates abstract, introduction, related work, methodology, experiments, limitations, conclusion, and data/code availability statements, while flagging unsupported performance claims and missing baselines. Venue configurations define page limits, column layouts, required sections, review models (single- or double-blind), and authoritative style templates. Table I summarizes the configured coverage. Decoupling rules into structured schemas ensures that supporting a new venue requires authoring a configuration file rather than modifying verification code.

B. Execution engine and project artifacts

The core engine operates as a state machine managing repository state. The state file tracks research topic, discipline, citation style, active phase, designated human approver, and logged overrides. Although Fig. 2 illustrates a canonical milestone sequence from ideation to final packaging, scholarly authoring is non-linear: authors and agents iteratively cycle between drafting, literature refinement, and experiments. CameraReady natively supports phase re-entry, issue reopening, and iterative refinement across all phases. Refinement markers protect finalized sections by causing modification scripts to abort if an overwrite is attempted.

VI. THE GATE MODEL

Fig. 2 illustrates the ten phases of the authoring workflow. Gate conditions are evaluated directly against physical repository files: `notes/research-question.md` must exist with non-empty content, the bibliography must meet the configured reference threshold without unresolved keys, and all issues CSV tasks must be completed with verified acceptance criteria.

When an agent requests a transition, the gate script evaluates the filesystem independently of model assertions. If a predicate

TABLE II
SYSTEM GATE INVENTORY BY FUNCTIONAL CATEGORY

Category	Gate Name	Target Invariant / Diagnostic	Type
Workflow	autopilot	Phase exit predicate holds	BLOCK
	kickoff	Plan approved before issues	BLOCK
	rebuttal gate	Review feedback itemized	BLOCK
Citation	verify citations	3-layer lookup + review	BLOCK
	citation style	Required schema metadata	BLOCK
	BibTeX audit	Duplicates, years, recency	BLOCK
	citation align	Sentence-abstract lexical overlap	ADVISE
Integrity	data gate	Numbers match regenerated data	BLOCK
	integrity gate	Claims, privacy, bibliography	BLOCK
	claim registry	Assertion extraction worksheet	ADVISE
	prose diagnostic	Repetitive n-gram pattern density	ADVISE
Venue / Ops	format gate	Page limit, sections, macros	BLOCK
	anonymity	Author macros, URLs, emails	BLOCK
	privacy scan	Sensitive secrets/keys in repo	BLOCK
	portability	Cross-platform script sanity	BLOCK
	venue rules	Sync with published CFP guide	BLOCK

fails, the gate exits with status code 1 and outputs the specific unmet prerequisite. To handle genuine research exceptions, the system supports a manual override: passing `--force` with a required justification advances the phase while appending an append-only record to the run journal:

```
forced past 'ideate' despite:
notes/research-question.md is missing.
Reason: demonstration of override journaling
```

Because journal entries are append-only through the CameraReady CLI state machine and cannot be purged via standard commands, deviations remain fully auditable. Table II summarizes the gate inventory across all functional categories.

VII. VERIFICATION MECHANISMS

A. The citation path

Fig. 3 details bibliographic ingestion. While an agent can formulate search queries, the CLI toolchain provides no programmatic path to inject raw BibTeX into `ref.bib`. Title queries are dispatched to DBLP and CrossRef, while identifier queries route to arXiv; retrieved records are normalized in a local SQLite registry before export. When resolving a query, `fetch_bibtex.py` exits with status 0, emitting an explicit [VERIFY] marker with zero BibTeX records upon lookup misses. Blocking enforcement occurs downstream: gates such as `verify_citations.py`, `integrity_gate.py`, and `arxiv_package.py` scan `ref.bib` and halt with exit code 1 upon detecting any unresolved [VERIFY] tag.

B. Resilience to external service disruptions

To survive API outages and rate limits, lookups are routed through a persistent local cache indexed by query hash. In network-isolated settings, users can configure an offline JSON Lines corpus of pre-indexed papers via `--offline`. Cached records strictly corroborate previously verified citations; unindexed queries in offline mode produce unverified markers rather than fabricated entries.

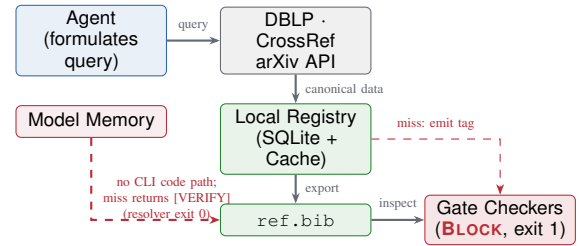


Fig. 3. Bibliographic dataflow and trust boundary: The dashed path from model memory to `ref.bib` is absent from the CLI toolchain. Query misses emit [VERIFY] tags (resolver exit 0), which are subsequently trapped by downstream blocking gates (exit 1).

C. Reconciling empirical data and reported metrics

To prevent numerical drift, the data gate links LaTeX tables and figures to data-generation scripts. The gate re-executes the script, extracts numerical tokens within the associated float, and confirms that reported numbers match regenerated outputs within numerical tolerance, excluding formatting numerals.

D. Semantic citation alignment

To catch contextual misattributions where a cited paper exists but does not support the citing claim, the alignment checker computes lexical and TF-IDF overlap between citing sentences and the cited abstract. Operating as an advisory diagnostic (**ADVISE**), low similarity scores flag citations for manual review without asserting automated judgment over conceptual validity.

E. Claim extraction, prose diagnostics, and packaging

The claim extractor parses manuscript prose into a structured registry of factual assertions for author review. The integrity gate requires explicit human sign-off on all claims before packaging. Prose diagnostics evaluate repetitive n-grams informatively without blocking compilation. The format gate validates venue page limits and sections, the anonymity checker flags author identifiers, and the packaging script bundles verified sources with inlined bibliographies for arXiv pre-submission.

VIII. SOFTWARE VERIFICATION AND SELF-APPLICATION

Evaluations were conducted on a Windows 10 workstation (build 10.0.19045, Python 3.13.9, MiKTeX). All reproduction commands and raw outputs are documented in `notes/experiment-results.md`. We evaluate CameraReady through software verification of its implementation (545 unit tests across 34 modules), synthetic negative control fault injections, an operational comparison of bibliographic provenance under live API conditions, and an end-to-end dogfooding self-application case study. Gate enforcement is evaluated under injected faults; running controlled comparison protocols against ungated agent baselines remains future work.

A. Unit verification and gate determinism

The test suite spans 545 unit tests across 34 modules, completing in 47.9 s (100% pass rate). Continuous integration validates every commit across a 3-OS (Ubuntu, macOS, Windows) $\times$ 2-Python-version matrix with CLI smoke tests.

TABLE III
NEGATIVE CONTROL INTERCEPTIONS UNDER FAULT INJECTION

Injected Fault Condition	Gate Diagnostic / Interception	Exit
Ideate without RQ	Flags missing research-question.md	1
Kickoff without approval	Halts: plan approval checkbox unmarked	1
Forced override w/ reason	Advances phase, logs rationale to journal	0
Missing IEEE journal	Rejects entry: mandatory key missing	1
Signed paper anonymity	Flags 2 author macros, 1 URL (3 blockers)	1
NeurIPS format breach	Flags missing sections and style	1
Unadjudicated claims	Halts packaging: claim registry unapproved	1
Fabricated paper title	Emits [VERIFY]; 0 BibTeX (resolver 0; gate 1)	0
Mismatched table numbers	Traps discrepancy between prose and data	1
Failing analysis script	Catches non-zero pipeline exit code	1
Untracked rebuttal point	Halts: unmapped reviewer item	1
Stale venue page limits	Catches outdated page count constraint	1

TABLE IV
THREE-LAYER BIBLIOGRAPHIC VERIFICATION STABILITY ACROSS SIX
REMOTE LOOKUP RUNS (SKIPPED DENOTES TRANSIENT REMOTE LOOKUP
THROTTLING, NOT HALLUCINATION; UNION REFLECTS DISTINCT ENTRIES
VERIFIED ACROSS RUNS).

Run	Verified	Suspicious	Hallucination Verdict	Skipped (Throttled / 429)
Run 1	39	0	0	6
Run 2	38	0	0	7
Run 3	41	0	0	4
Run 4	38	0	0	7
Run 5	45	0	0	0
Run 6	45	0	0	0
Union (1–6)	45	0	0	0 (complete)

The portability checker verified zero OS-specific assumptions across 218 source files, and the specification validator reported zero configuration warnings. Across the framework’s 68 scripts, 62 implement fail-stop non-zero exit codes; the worked example compiles in 5.8 s.

B. Gate enforcement under negative controls

We evaluated gate enforcement against twelve synthetic negative control fixtures representing intentional violations (Table III). In all blocking cases, the gate intercepted non-compliant artifacts and exited with status 1. Tested with a fabricated title (“Gated Diffusion Transformers for Bibliographic Provenance Repair”), the query emitted an unresolved marker with zero BibTeX records. The data gate similarly intercepted injected discrepancies (0.88 reported vs. 0.71 produced).

C. Heuristic checker refinements

Early claim extractors tokenized raw LaTeX markup into 48 noisy claims (including a vector graphic); an AST cleaning pre-pass reduced this to 26 substantive assertions. Likewise, transitioning prose logic checks to whole-document TF-IDF cosine similarity reduced false-positive topical disconnect warnings from 14 to 1 on this manuscript.

D. Provenance by construction vs. post-hoc screening

In this case study, all 45 cited references were resolved via canonical arXiv queries (recorded with their persistent identifiers in `citation-provenance.jsonl`) and exported into `ref.bib`. To test post-hoc consistency, we executed the three-layer verifier across six afternoon runs under normal network conditions (Table IV). Under the stated registry

criterion, no identity-level fabricated bibliographic records were observed. However, verified entries fluctuated between 38 and 45 due to transient upstream throttling (recorded in `notes/experiment-results.md`). Tested with three fabricated citations lacking identifiers, the verifier returned skipped rather than hallucination verdicts. While declining to guess is sound, inconclusive verdicts provide no safety guarantee. Persisting canonical provenance in SQLite at ingestion avoids re-resolution volatility and shields workflows from remote outages.

E. Dogfooding self-application case study

This manuscript was produced end-to-end using CameraReady as a dogfooding case study through ideation, literature gathering, plan approval, issues tracking (expanding from 38 to 45 tasks), experiments, and drafting. During review, the rebuttal gate intercepted five unaddressed comments, automatically mapped to tracked issues via `--emit-issues`. One manual override was exercised during drafting when 16 optional presentation tasks were skipped; the transition and rationale were permanently recorded in the run journal.

IX. DISCUSSION, LIMITATIONS, AND FUTURE WORK

Validity, trust, and bounds. CameraReady enforces syntactic and bibliographic invariants; evaluating scientific novelty or conceptual validity remains the author’s responsibility [14]. The framework assumes a cooperative agent; isolation against untrusted shell agents requires OS-level sandboxing. Textual heuristics provide diagnostic assistance rather than definitive verdicts, while local SQLite caching mitigates external API dependencies.

Controlled benchmarking and future work. While this report verifies gate correctness under synthetic fault injection, controlled empirical benchmarking of authoring error rates against unconstrained baselines (formalized in `benchmark_compare.py`) remains future work, alongside neural entailment for citation alignment and containerized execution recipes.

X. CONCLUSION

Citation hallucination, numerical drift, and formatting violations are structural vulnerabilities of unconstrained authoring agents. CameraReady demonstrates that mechanically checkable manuscript invariants can be deterministically enforced by embedding verification gates directly into the authoring lifecycle. Decoupling parametric memory from the bibliography, conditioning phase transitions on filesystem predicates, and formalizing venue rules as schemas remove the supported CLI path for injecting unverified bibliographic records while preserving transparent boundaries for advisory heuristics and scholarly human judgment. Controlled empirical comparisons against unconstrained agent baselines remain future work.

DATA AND CODE AVAILABILITY

The complete system is open source (MIT license) at <https://github.com/e96031413/camera-ready> (tag `v0.2.0`). Reproduction commands are in `notes/experiment-results.md`. Provenance records for all 45 references (canonical IDs, timestamps, and SHA-256 hashes) are in `citation-provenance.jsonl`.

REFERENCES

- [1] Z. Zhao, Y. Wang, T. Stuart, M. D. Vaan, P. Ginsparg, and Y. Yin, "Llm hallucinations in the wild: Large-scale evidence from non-existent citations," 2026.
- [2] M. Russinovich, R. S. S. Kumar, and A. Salem, "Phantom references: Hallucinated citations that survive peer review at top-tier conferences," 2026.
- [3] M. Naser, "How llms cite and why it matters: A cross-model audit of reference fabrication in ai-assisted academic writing and methods to detect phantom citations," 2026.
- [4] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha, "The ai scientist: Towards fully automated open-ended scientific discovery," 2024.
- [5] S. Schmidgall, Y. Su, Z. Wang, X. Sun, J. Wu, X. Yu, J. Liu, M. Moor, Z. Liu, and E. Barsoum, "Agent laboratory: Using llm agents as research assistants," 2025.
- [6] M. Zhao, J. Dong, K. Xu, Z. Hasan, C. Fan, S. Jiang, S. Mao, Y. Ling, L. Zou, T. Zhou, Y. H. Chan, W. Zhang, Z. Zhou, G. Huang, H. Li, W. Cun, Z. Chen, M. Yuan, and Y. Geng, "Scienceflow: A long-horizon agent for ml research, scientific discovery and beyond," 2026.
- [7] L. Mitchener, A. Yiu, B. Chang, M. Bourdenx, T. Nadolski, A. Sulovari, E. C. Landsness, D. L. Barabasi, S. Narayanan, N. Evans, S. Reddy, M. Foiani, A. Kamal, L. P. Shriver, F. Cao, A. T. Wassie, J. M. Laurent, E. Melville-Green, M. Caldas, A. Bou, K. F. Roberts, S. Zagorac, T. C. Orr, M. E. Orr, K. J. Zvezdaryk, A. E. Ghareeb, L. McCoy, B. Gomes, E. A. Ashley, K. E. Duff, T. Buonassisi, T. Rainforth, R. J. Bateman, M. Skarlinski, S. G. Rodrigues, M. M. Hinks, and A. D. White, "Kosmos: An ai scientist for autonomous discovery," 2025.
- [8] Q. Tang, T. Sun, X. Hu, X. Liu, Y. Chen, Y. Shao, B. Li, C. Lv, C. Xu, C. Huang, C. Li, D. Xue, H. Bai, H. Duan, H. Guo, H. He, H. Chen, H. Shen, J. Yuan, J. Sun, J. Cheng, J. Xu, J. Tong, J. Chen, J. Liu, J. Leng, J. Yu, K. Jiang, K. Xiang, K. Yao, L. Feng, L. Yuan, L. Gao, M. Li, Q. Jia, Q. Sun, S. Ding, S. Gong, S. Zhong, T. J. Zhang, T. Gu, T. Liang, W. Liu, W. Yang, W. Fei, X. Wang, X. Liu, X. Ding, Y. Tang, Y. Wang, Y. Jiang, Y. Yang, Z. He, Z. Chen, Z. Xu, Z. Li, and Z. Huang, "Fars: A fully automated research system deployed at scale," 2026.
- [9] Y. Du, B. Yu, T. Liu, T. Shen, J. Chen, J. G. Rittig, K. Sun, Y. Zhang, A. Krishnan, Y. Zhang, D. Rosen, R. Pirone, Z. Song, B. Zhou, C. Masschelein, Y. Wang, H. Wang, H. Jia, C. Zhang, H. Zhao, M. Ester, N. Hacohen, T. Head-Gordon, C. P. Gomes, H. Sun, C. Duan, P. Schwaller, and W. Jin, "Accelerating scientific discovery with autonomous goal-evolving agents," 2026.
- [10] Z. Qian, W. Shi, X. Lin, L. Ling, M. Luo, Z. Wang, Z. Zhang, T. Xu, G. Liu, Z. Zhang, S. Zhang, Z. Wang, Z. Feng, Y. Luo, S. Xu, Y. Chen, Z. Feng, Z. Chen, B. Yuan, B. Wu, H. Wang, and K. Chen, "Story2proposal: A scaffold for structured scientific paper writing," 2026.
- [11] S. Ren, C. Xie, P. Jian, Z. Ren, C. Leng, and J. Zhang, "Towards scientific intelligence: A survey of llm-based scientific agents," 2026.
- [12] G. Tie, P. Zhou, and L. Sun, "A survey of ai scientists," 2026.
- [13] L. Kong, X. Sun, W. Chow, L. Li, K. Q. Lin, X. B. Zhang, S. Wang, R. Li, Q. Wu, W. Gao, Y. Wang, S. Xie, J. Liu, L. Qu, S. Li, L. X. Ng, B. R. Cottreau, Z. Liu, T.-S. Chua, and W. T. Ooi, "Ai for auto-research: Roadmap & user guide," 2026.
- [14] A. Miyai, M. Toyooka, Z. Zhao, K. Watanabe, T. Yamasaki, and K. Aizawa, "Paper reconstruction evaluation: Evaluating presentation and hallucination in ai-written papers," 2026.
- [15] D. Rao, E. Wong, and C. Callison-Burch, "Detecting and correcting reference hallucinations in commercial llms and deep research agents," 2026.
- [16] K. Khajavi, S. Sadeghi, R. Adhikari, and A. Tessier, "Citecheck: Retrieval-grounded detection of llm citation hallucinations in scientific text," 2026.
- [17] C. Cornelio, T. Ito, R. Cory-Wright, S. Dash, and L. Horesh, "The need for verification in ai-driven scientific discovery," 2025.
- [18] I. Takahara and T. Mizoguchi, "Toward auditable ai scientists: A hypothesis evolution protocol for llm agents," 2026.
- [19] Y. Wang, "Firstresearch: Auditable question formation for llm scientific discovery agents," 2026.
- [20] Z. Wang, H. Li, Z. Yang, Z. Hu, S. Zuo, Y. Zhang, D. Ma, D. Luo, C. Wang, J. Peng, T. Huang, S. Guo, H. Wang, Z. Zhu, S. Han, Y. Cao, B. Chen, X. Chen, K. Yu, and L. Chen, "Externalizing research synthesis and validation in ai scientists through a research harness," 2026.
- [21] Y. Chen, Y. Quan, X. Lin, and R. Tang, "Where fake citations are made: Tracing field-level hallucination to specific neurons in llms," 2026.
- [22] M. Dassen, R. Kotula, K. Murray, A. Yates, D. Lawrie, E. Kayi, J. Mayfield, and K. Duh, "Factum: Mechanistic detection of citation hallucination in long-form rag," 2026.
- [23] D. Abbonato, "Checkifexist: Detecting citation hallucinations in the era of ai-generated content," 2026.
- [24] M. Li, Z. Lin, and S. Ma, "Source or it didn't happen: A multi-agent framework for citation hallucination detection," 2026.
- [25] D. Rao and C. Callison-Burch, "Bibtex citation errors in scientific publishing agents: Evaluation and mitigation," 2026.
- [26] J. Lee, "citecheck: An mcp server for automated bibliographic verification and repair in scholarly manuscripts," 2026.
- [27] P. Bécharde and O. M. Ayala, "Reducing hallucination in structured outputs via retrieval-augmented generation," 2024.
- [28] Y. Liu, W. Zhang, Z. Yang, Z. Zhang, H. Feng, X. Wang, P. Qiu, Y. Liu, B. Poczos, and J. B. Hong, "Care: Pre-execution command verification for shell-executing llm agents," 2026.
- [29] B. Yu, C. Jia, J. Chi, X. Liu, Y. Wang, H. Bai, Y. Liu, J. Wei, and J. Zhu, "Tracer: Verifiable generative provenance for multimodal tool-using agents," 2026.
- [30] Y. She, Y. Liang, and E. Kang, "Safeguarding llm agents from misalignment through provenance analysis," 2026.
- [31] R. Souza, T. Poteet, B. Etz, D. Rosendo, A. Gueroudji, W. Shin, P. Balaprakash, and R. F. da Silva, "Llm agents for interactive workflow provenance: Reference architecture and evaluation methodology," 2025.
- [32] Y. Wu, S. Shen, R. Yang, H. Peng, and B. Hu, "Verify, repair, repeat, or stop? robust stopping for noisy verify-repair loops in llm agents," 2026.
- [33] T. Sah, V. Srivastava, D. Sah, and K. Jordan, "The verifier tax: Horizon dependent safety success tradeoffs in tool using llm agents," 2026.
- [34] H. Xu, C. Li, X. Ma, X. Ou, Z. Zhang, T. He, X. Liu, Z. Wang, J. Liang, Z. Chu, R. Liu, R. Mu, D. Tu, M. Liu, and B. Qin, "The evolution of tool use in llm agents: From single-tool call to multi-tool orchestration," 2026.
- [35] H. Liu, Y. Ming, S. Joty, and C. Zhao, "Harnessing llm agents with skill programs," 2026.
- [36] R. Ye, X. Pang, J. Chai, J. Chen, Z. Yin, Z. Xiang, X. Dong, J. Shao, and S. Chen, "Are we there yet? revealing the risks of utilizing large language models in scholarly peer review," 2024.
- [37] A. P. Akella, H. V. Siravuri, and S. Rohatgi, "Pre-review to peer review: Pitfalls of automating reviews using large language models," 2025.
- [38] L. Zhou, R. Zhang, X. Dai, D. Hershovich, and H. Li, "Large language models penetration in scholarly writing and peer review," 2025.
- [39] W. Wu, C. Zhang, Y. Zhao, and T. Bao, "Impact of large language models on peer review opinions from a fine-grained perspective: Evidence from top conference proceedings in ai," 2026.
- [40] Z. Yin and S. Wang, "Span-level detection of ai-generated scientific text via contrastive learning and structural calibration," 2025.
- [41] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research (a report from the neurips 2019 reproducibility program)," 2020.
- [42] E. K. Akdeniz, S. Tekir, and M. N. A. A. Hinnawi, "An end-to-end system for reproducibility assessment of source code repositories via their readmes," 2023.
- [43] Y.-H. Huang and Y.-S. Lin, "Lumen: Cost-transparent multi-agent pipeline for automated systematic review and meta-analysis," 2026.
- [44] H.-T. Lin and J.-T. Yeh, "meta-pipe: An llm-agent pipeline for end-to-end automated systematic review and meta-analysis," 2026.
- [45] X. Chen and X. Zhang, "Large language models streamline automated systematic review: A preliminary study," 2025.